

Introduction to Pointers

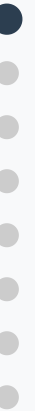


Understanding the powerful concept of pointers in C programming



Why Learn Pointers?

If you want to be proficient in writing of code in C programming language, you must have a thorough working knowledge of how to use pointers. One of those things, beginners in C find difficult is the concept of pointers. The purpose of this unit is to provide an introduction to pointers and their efficient use in C programming. Actually, main difficulty lies with C's pointer terminology than actual concept.





Dynamic Data Structures

C uses pointers to create data structures built up from blocks of memory allocated from heap at run-time



Variable Parameters

C uses pointers to handle variable parameters passed to functions



Array Access

Pointers provide an alternative means of accessing information stored in arrays, especially valuable with strings

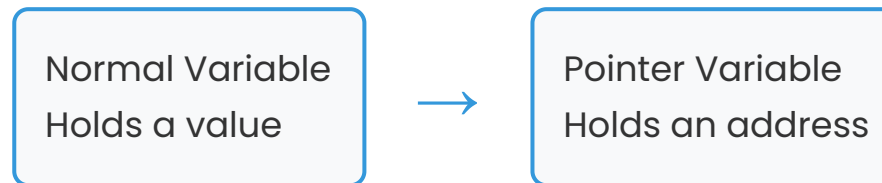
Pointers vs Normal Variables

A normal variable is a location in memory that can hold a value. For example, when you declare a variable `i` as an integer, four bytes of memory is set aside for it. In your program, you refer to that location in memory by name `i`. At the machine level, that location has a memory address, at which four bytes can hold one integer value.



What is a Pointer?

A pointer is a variable that points to another variable. This means that it holds the memory address of another variable. Put another way, pointer does not hold a value in traditional sense; instead, it holds address of another variable. It points to that other variable by holding its address.



Accessing Memory Addresses

Because a pointer holds an address rather than a value, it has two parts. The pointer itself holds address. That address points to a value. There is pointer and value pointed to. As long as

you're careful to ensure that pointers in your programs always point to valid memory locations, pointers can be useful, powerful, and relatively trouble-free tools.



Example: Printing Memory Addresses

```
#include<stdio.h>
int main ()
{
    int var1;
    char var2[10];
    printf("Address of var1 variable: %x\n", &var1 );
    printf("Address of var2 variable: %x\n", &var2 );
    return 0;
}
```

```
Address of var1 variable: bff5a400
Address of var2 variable: bff5a3f6
```

Defining Pointers

A pointer is a variable whose value is address of another variable, i.e., direct address of memory location. Like any variable or constant, you must declare a pointer before using it to store any variable address.



Pointer Declaration Syntax

```
Type *var-name;
```

Here, type is pointer's base type; it must be a valid C data type and var-name is name of pointer variable. The asterisk * used to declare a pointer is the same asterisk used for multiplication. However, in this statement, asterisk is being used to designate a variable as a pointer.

Pointer Declarations with Different Data Types

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.



Valid Pointer Declarations

```
int *ip; /* pointer to an integer */
double *dp; /* pointer to a double */
float *fp; /* pointer to a float */
char *ch; /* pointer to a character */
```



Integer Pointer

Points to integer values



Character Pointer

Points to character values



Float/Double Pointer

Points to floating-point
values

Concept of Pointer Variables

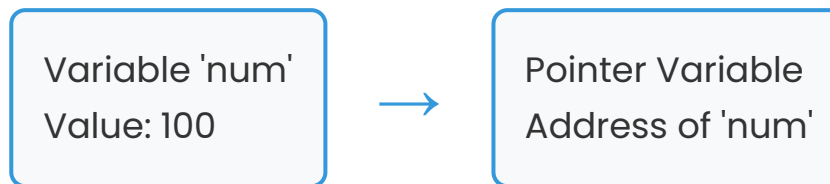
An ordinary variable is a location in memory that can hold a value. For example, when you declare a variable `num` as an integer, the compiler sets aside 2 bytes of memory (depends on PC) to hold the value of the integer. In your program, you refer to that location in memory by the name `num`. At the machine level, that location has a memory address.



Example Variable

```
int num = 100;
```

We can access the value 100 either by the name num or by its memory address. Since addresses are simply digits, they can be stored in any other variable. Such variables that hold addresses of other variables are called Pointers.



Characteristic Features of Pointers

With the use of pointers in programming:

- The program execution time will be faster as data is manipulated with the help of addresses directly
- Will save memory space

- The memory access will be very efficient
- Dynamic memory is allocated



Faster Execution

Direct memory access speeds up program execution



Memory Efficient

Saves memory space by using addresses



Dynamic Allocation

Enables dynamic memory allocation

Thorough knowledge of pointers is very much essential for your future courses like data structures, design and analysis of algorithms, etc.